

Data on the Line · Prep note

Demand forecasting and line balancing · about 20 minutes to read

This note introduces the ideas you'll use in the case, with small examples that have nothing to do with it. The numbers in the examples are made up. Reading it and running the setup test before the session is all the preparation you need.

1. Takt time

Takt time tells you how often a unit has to come off the line to keep up with demand:

$$\text{takt time} = \text{available production time} / \text{units to be made}$$

Available time is what's left of the shift once breaks, planned maintenance and stoppages are taken out. With 50,400 seconds of net working time a day and 600 units to make each day, the takt time is 84 seconds. If any station needs more than 84 seconds per unit, the line can't keep that pace.

In lean manufacturing, takt is worked out from customer demand alone. Once the number of units also includes stock top-ups or replacements for scrapped units, the result is strictly a **planned cycle time**. In practice, both are often just called takt.

2. Assembly line balancing

On an assembly line, each unit passes through the stations in turn, and some of the tasks are done at each station. **Line balancing** is the problem of assigning tasks to stations. The key terms:

- **Task:** the smallest indivisible unit of work; each task has a duration.
- **Precedence relation:** some tasks can't start until others are finished. A task can never sit at an earlier station than a task that has to come before it.
- **Station load:** the total time of the tasks assigned to a station.
- **Bottleneck:** the station with the highest load. It sets the real pace of the line; a line can only run as fast as its busiest station.

The literature distinguishes two basic versions of the problem:

- **Type 1:** the takt time is given; the aim is to **minimise the number of stations** without any station's load exceeding the takt time.
- **Type 2:** the number of stations is given; the aim is to **minimise the load of the busiest station**, i.e. the cycle time of the line.

You can work out a **lower bound** on the number of stations:

$$\text{lower bound} = \lceil \text{total work content} / \text{takt time} \rceil$$

With 410 seconds of work and an 84-second takt, $410 / 84 = 4.88$, so you need at least 5 stations. The lower bound proves that fewer stations can't work, but it doesn't guarantee that this many will. Tasks can't be split, and precedence relations and other rules may get in the way.

You can measure how well a line is balanced with its **line efficiency**:

$$\text{line efficiency} = \text{total work content} / (\text{number of stations} \times \text{takt time})$$

The closer it is to 100%, the less idle time there is at the stations.

Mixed-model lines. When several products share a line, task times depend on the product. Each task then gets a **weighted average time** based on the product mix: if a product makes up 40% of production, its time counts for 40% of the average. But a station that fits within takt on average can still run well over takt for an individual product. On real lines, short overruns like these are absorbed by buffers and by how products are sequenced, but how much overrun is acceptable has to be checked separately.

3. Mixed-integer linear programming (MILP)

Line balancing problems are usually solved with **mixed-integer linear programming** (MILP). A MILP model has three parts:

- **Decision variables:** for example, 0–1 (binary) variables that answer "is task i at station k ?"
- **Objective function:** a single expression to minimise or maximise.
- **Constraints:** the rules the solution has to follow.

"Linear" means that the objective and the constraints are sums of decision variables multiplied by constants. Multiplying two variables together, or squaring one, is not linear.

Two modelling techniques worth knowing:

- **Minimising the largest value:** "the load of the busiest station" isn't a linear expression in itself. Instead, you add an auxiliary variable, constrain every station's load to be no more than it, and minimise that variable.
- **Two goals in a strict order:** if one goal takes strict priority over another, you can combine them in a single objective. Give the first goal a weight so large that no improvement in the second goal can ever make up for a one-unit loss in the first. It's enough for the weight to be larger than the largest value the second goal can take.

A MILP solver finds an optimal solution and, given enough time, proves that it is optimal. **Heuristics** are fast, but they can't guarantee that what they find is optimal.

The example below has nothing to do with line balancing: three jobs are assigned to two machines so that the busier machine's load is as small as possible. But it uses every PuLP construct you'll need in the case: 0–1 variables indexed by two sets, sums, an objective, constraints and the auxiliary-variable technique above.

```
import pulp

JOBS, MACHINES = ["A", "B", "C"], [1, 2]
HOURS = {"A": 4, "B": 3, "C": 2}

m = pulp.LpProblem("example", pulp.LpMinimize)
# x[j][k] = 1: job j runs on machine k
x = pulp.LpVariable.dicts("x", (JOBS, MACHINES), cat="Binary")
# load of the busier machine (hours)
longest = pulp.LpVariable("longest", lowBound=0)

# objective: keep the busier machine as light as possible
m += longest
# every job on exactly one machine
for j in JOBS:
    m += pulp.lpSum(x[j][k] for k in MACHINES) == 1
# no machine's load above longest
for k in MACHINES:
    m += pulp.lpSum(HOURS[j] * x[j][k] for j in JOBS) <= longest

m.solve(pulp.PULP_CBC_CMD(msg=False))
plan = {j: k for j in JOBS for k in MACHINES if x[j][k].value() > 0.5}
print(pulp.LpStatus[m.status], plan)
```

The example is written for PuLP version 3. The version 4 that comes with Colab changed this syntax, so on Colab first run `!pip install -q "pulp>=2.7,<4"` in a separate cell. If you get an error such as "Bad CPU type" on

your own Mac, install the HiGHS solver with `pip install highspy` and, in the `m.solve(...)` line, replace `pulp.PULP_CBC_CMD(msg=False)` with `pulp.HiGHS(msg=False)`. The case notebook does both of these for you.

4. Measuring forecast error

Two measures are used together:

$$\text{WAPE} = \sum |\text{actual} - \text{forecast}| / \sum \text{actual}$$

$$\text{Bias} = \sum (\text{forecast} - \text{actual}) / \sum \text{actual}$$

WAPE tells you how far off you were, as a share of total demand. Bias tells you the direction: positive means you systematically forecast too high, negative too low.

An example: actual demand over three months is 100, 120 and 80, and the forecasts are 110, 110 and 90. The absolute errors add up to $10 + 10 + 10 = 30$ and total demand is 300, so WAPE is 10%. The signed errors add up to $+10 - 10 + 10 = +10$, so bias is +3.3%. Read the two together: a model with a low WAPE that always misses in the same direction still has a problem.

5. Testing a forecasting model

You find out whether a model is any good by how well it forecasts **data it hasn't seen**, not the data it was trained on. So the data are split into a training part and a test part: the model is built on the first and evaluated on the second. How you split them should suit the structure of the data and the way the model will actually be used.

Every model should be compared with a simple **benchmark**. "Same month last year" (the seasonal naive method) is a common one. A model that can't beat the benchmark isn't worth using, however sophisticated it is.